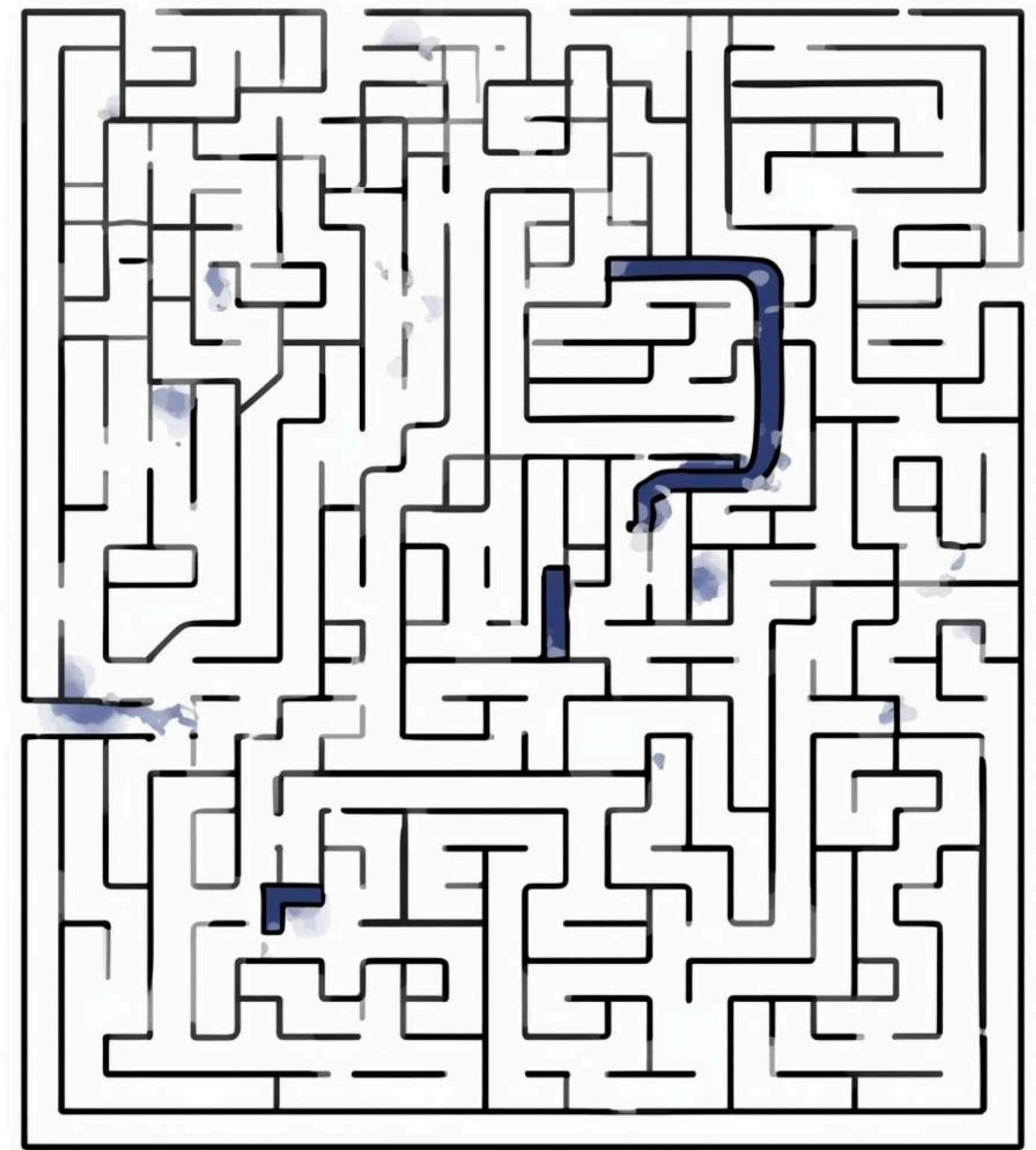


OOP 期末專題

Frozen Lake 強化學習專案 (Part 2)

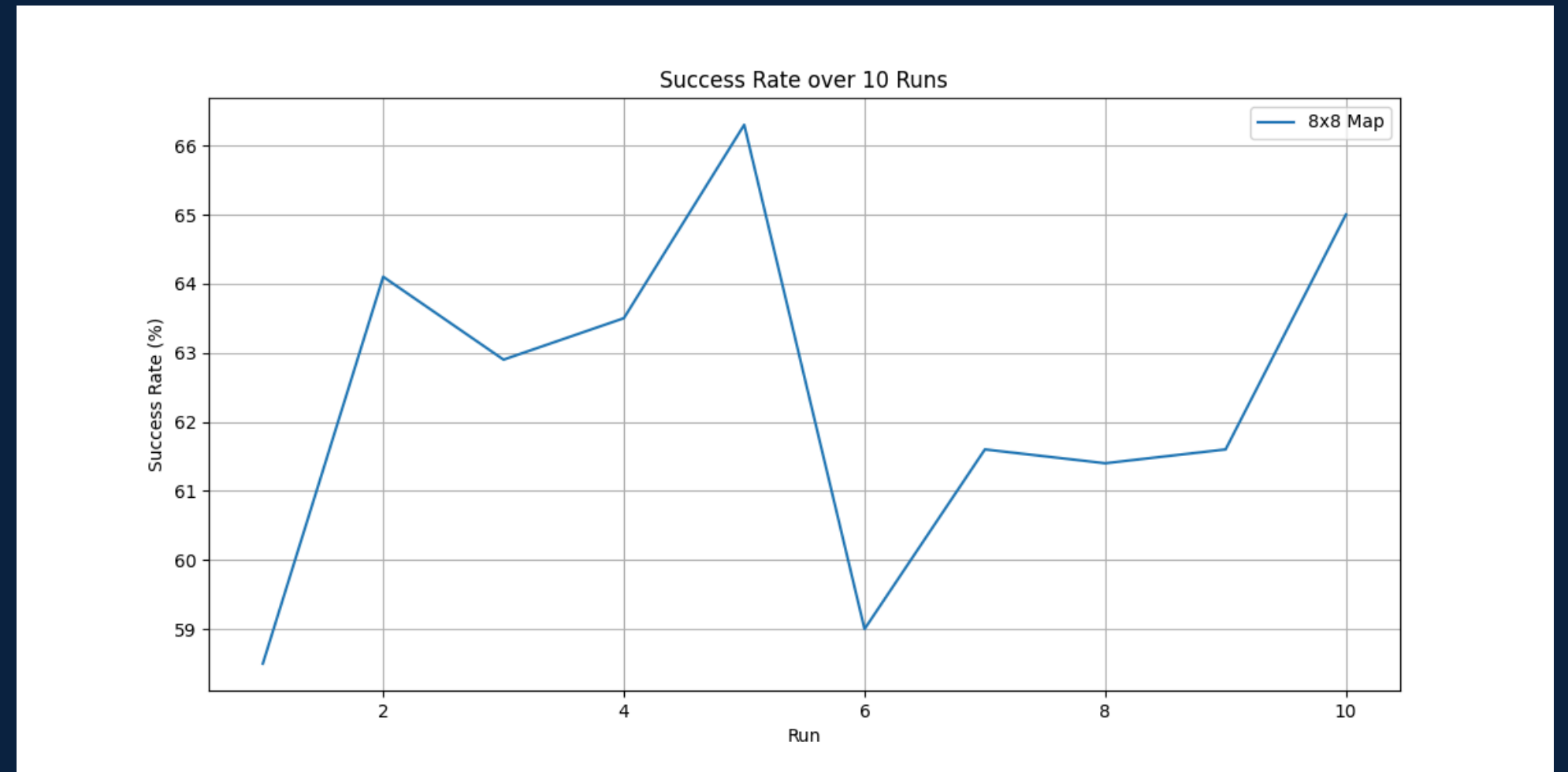
Dungeon Crawler 強化學習專案 (Part 3)



Part2

做了哪些改動

結果: 8x8 Slippery 地圖測試, 成功率約 63% (多次 runs 平均)



- 定義額外獎懲:
 1. 步數懲罰($\text{step_penalty}=0.00035$)
 2. 接近終點加分($\text{closer_to_goal_reward}=0.005$)
- 用轉移機率計算總期望值加速收斂
- 風險過濾: 每個動作不超過風險門檻($\text{RISK_THRESHOLD} = 0.45$)
- 避免後退: 每個動作倒退一格的機率不大於 ($\text{BACKTRACK_THRESHOLD} = 0.35$)

Part3

專案目標

- 拿鑰匙 → 開門 → 拿寶藏
- 避開陷阱/敵人
- training → saved model → testing
- Demo: Human 規則 + Agent 測試

Reward 設計:

1. 基礎: -0.1 (每步懲罰, 鼓勵快速完成)
2. 撞牆: -5.0 (避免卡住)
3. 找到鑰匙: +50
4. 開門: +10
5. 靠近目標: +0.1
6. 遠離目標: -0.1
7. 踩到陷阱: -hp_loss
8. 贏得遊戲: +100
9. 死亡: -50



三層架構

讓系統更容易維護

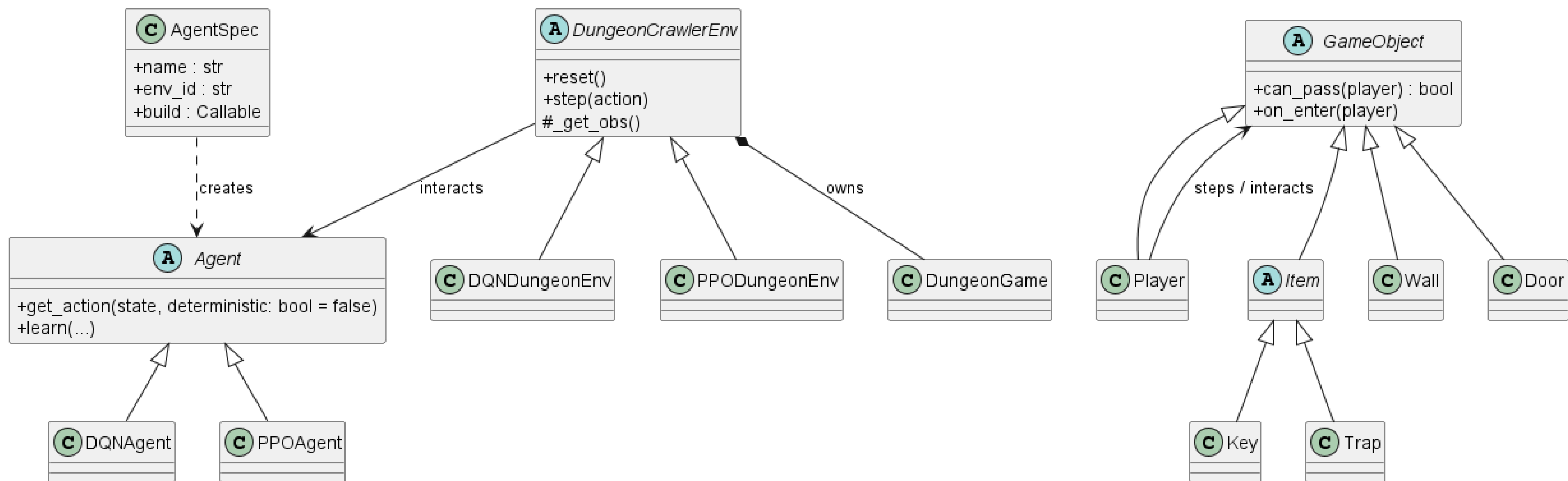
GameObject: 負責互動行為

Env: 負責流程控制

Agent: 負責決策

UML 類別圖

(簡化版)



Agent: Q-learning

Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode):

Initialize s

Repeat (for each step of episode):

Choose a from s using policy derived from Q

Take action a , observe r, s'

Update

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$s \leftarrow s'$;

Until s is terminal

Agent: DQN

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N

Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

 With probability ϵ select a random action a_t

 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

end for

end for

Agent: DDQN

Algorithm 1 Double Q-learning

```
1: Initialize  $Q^A, Q^B, s$ 
2: repeat
3:   Choose  $a$ , based on  $Q^A(s, \cdot)$  and  $Q^B(s, \cdot)$ , observe  $r, s'$ 
4:   Choose (e.g. random) either UPDATE(A) or UPDATE(B)
5:   if UPDATE(A) then
6:     Define  $a^* = \arg \max_a Q^A(s', a)$ 
7:      $Q^A(s, a) \leftarrow Q^A(s, a) + \alpha(s, a) (r + \gamma Q^B(s', a^*) - Q^A(s, a))$ 
8:   else if UPDATE(B) then
9:     Define  $b^* = \arg \max_a Q^B(s', a)$ 
10:     $Q^B(s, a) \leftarrow Q^B(s, a) + \alpha(s, a) (r + \gamma Q^A(s', b^*) - Q^B(s, a))$ 
11:   end if
12:    $s \leftarrow s'$ 
13: until end
```

Agent: PPO

PPO algorithm

$$J_{PPO}^{\theta^k}(\theta) = J^{\theta^k}(\theta) - \beta KL(\theta, \theta^k)$$

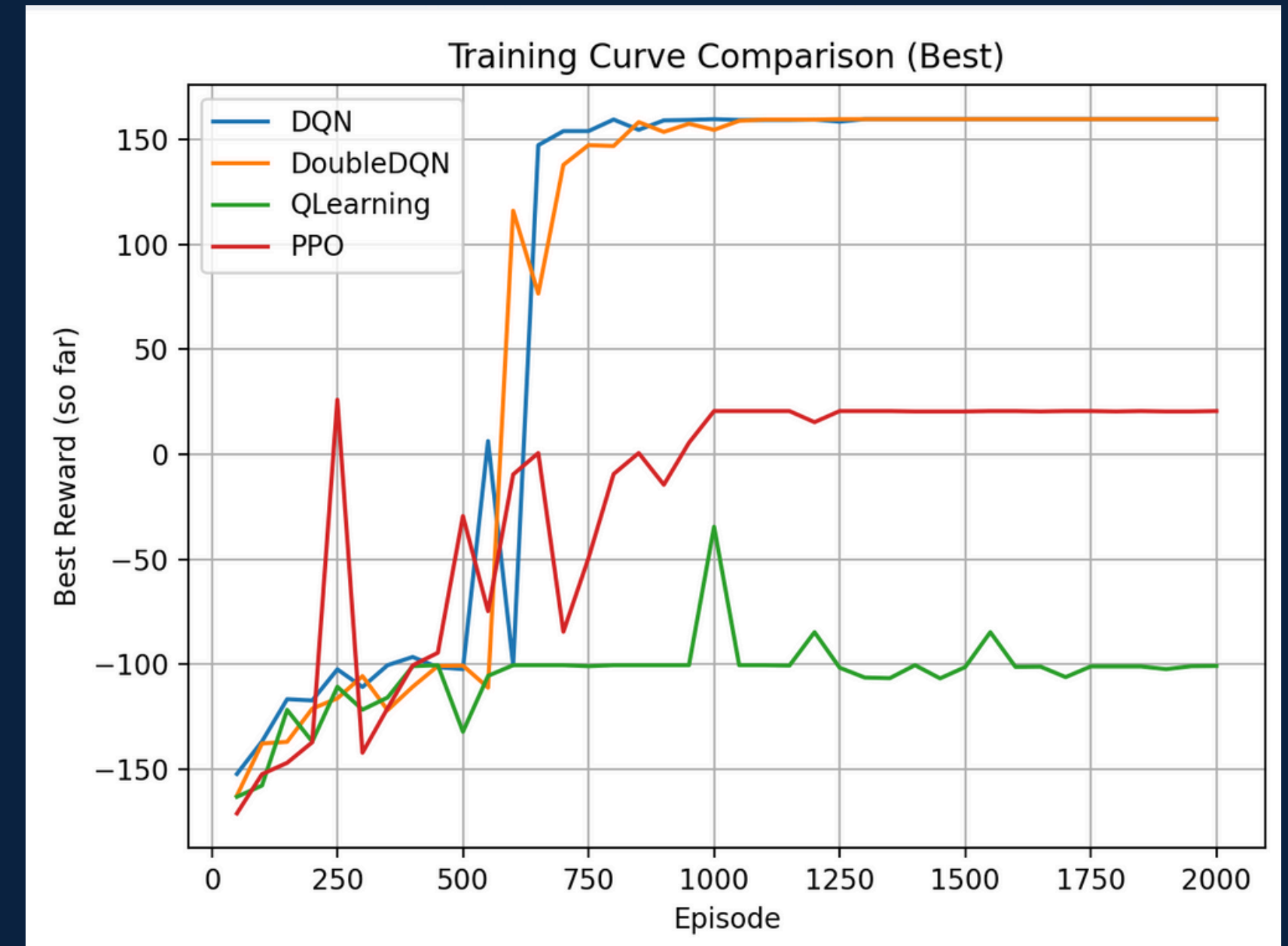
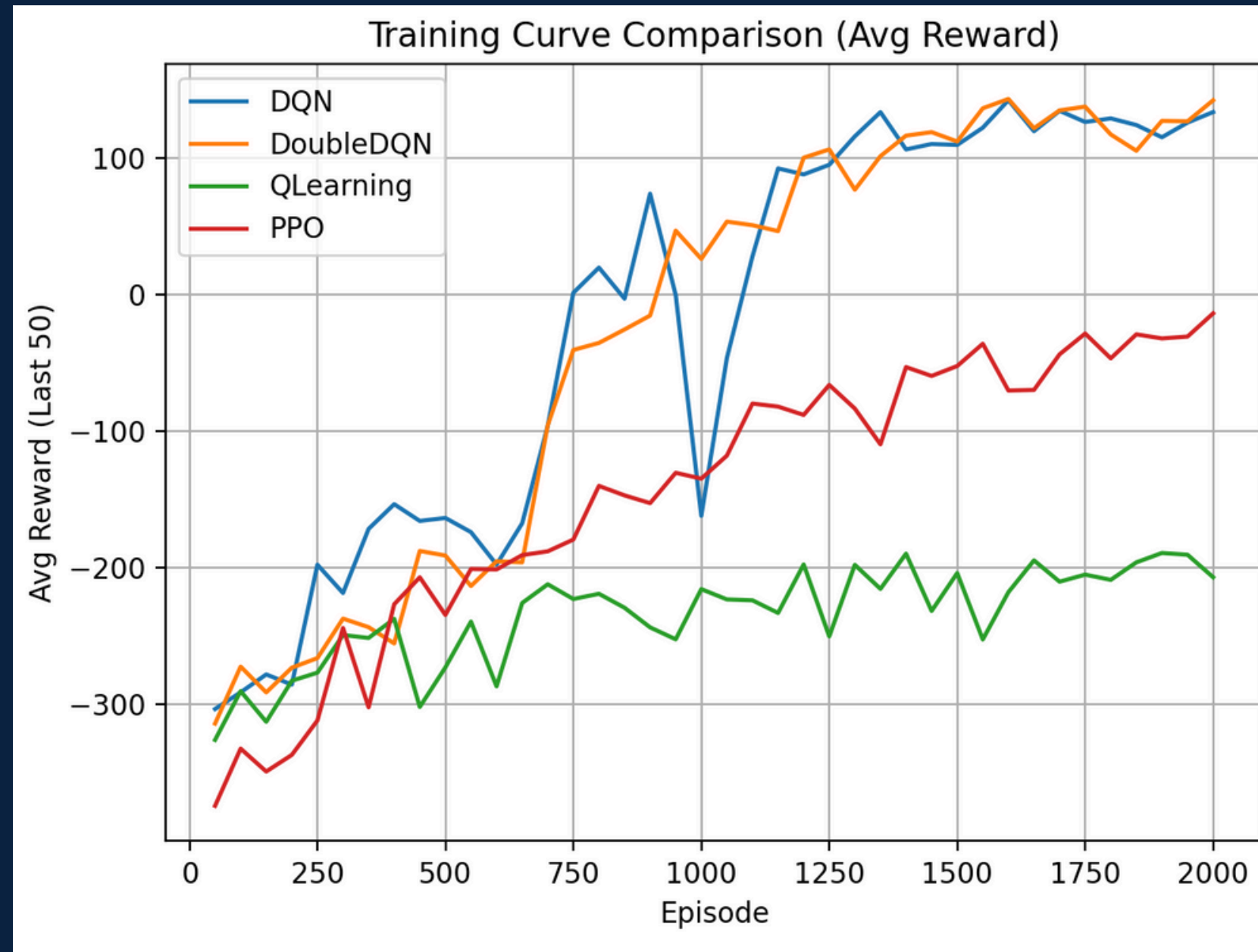
$$J^{\theta^k}(\theta) \approx \sum_{(s_t, a_t)} \frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)} A^{\theta^k}(s_t, a_t)$$

PPO2 algorithm

$$J_{PPO2}^{\theta^k}(\theta) \approx \sum_{(s_t, a_t)} \min \left(\frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)} A^{\theta^k}(s_t, a_t), \right.$$

$$\left. \text{clip} \left(\frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)}, 1 - \varepsilon, 1 + \varepsilon \right) A^{\theta^k}(s_t, a_t) \right)$$

Some Experimental Result



Q&A

